



COMPRESOR DE PARTIDAS DE AJEDREZ (CPG5)

Práctica final de CDI

Autores:

Alex Lafuente Gonzalez

Daniel Mejias Cerezo

Adrià Cebrián Ruiz

Facultat Informàtica de Barcelona, UPC

31 de mayo de 2026

Índice

1. Introducción	2
2. Metodología de Compresión	2
2.1. Codificación de Cabeceras (Headers)	2
2.1.1. Codificación estructurada por tipo de campo	2
2.1.2. Mecanismos de escape y robustez	4
2.1.3. Codificación de nombres de jugadores	4
2.2. Codificación de Movimientos (Chess Logic)	5
2.2.1. Empaquetado de Plies	5
2.2.2. Codificación del Resultado	5
2.2.3. Tratamiento de Anotaciones	6
2.2.4. Elementos Ignorados	7
2.2.5. Estructura Binaria de una Partida	7
2.2.6. Formato Global CPG5	7
2.3. Paralelización	8
2.3.1. Separación estricta de fases	8
2.3.2. Modelo de paralelización	8
2.3.3. Ejecución segura y tolerancia a fallos	8
2.3.4. Reensamblaje determinista	9
2.3.5. Optimización de escritura	9
2.3.6. Justificación del diseño híbrido	9
2.4. Estructura del Binario CPG5	9
2.4.1. Propiedades estructurales del formato	11
2.4.2. Invarianza del formato	11
3. Resultados	11
4. Conclusiones	11
Referencias	12

1. Introducción

El formato Portable Game Notation (PGN) es el estándar universalmente aceptado para el almacenamiento e intercambio de partidas de ajedrez. Sin embargo, al tratarse de un formato basado puramente en texto plano, presenta un grado de redundancia sumamente elevado. Las cabeceras repiten constantemente etiquetas estáticas (como *Event*, *Site*, *White*, *Black*), y los movimientos se registran utilizando notación algebraica, ignorando por completo la lógica intrínseca del juego.

Para resolver este problema, hemos diseñado e implementado el formato **CPG5**, un compresor avanzado de partidas de ajedrez. A diferencia de compresores de propósito general (como ZIP o GZIP) que tratan el PGN como un flujo de bytes opacos, CPG5 es un *compresor consciente del dominio*. Extrae la semántica del ajedrez para codificar cada aspecto de la partida de forma óptima: utilizando diccionarios dinámicos, códigos Huffman basados en frecuencias reales extraídas de un gran corpus, codificación diferencial temporal y, lo más importante, emulación del estado del tablero para almacenar únicamente el índice del movimiento legal escogido en cada turno.

2. Metodología de Compresión

El compresor divide el proceso en dos grandes dominios: las cabeceras y los movimientos de las partidas.

2.1. Codificación de Cabeceras (Headers)

Cada tag del header se identifica mediante un prefijo de 5 bits, lo que permite representar hasta 32 etiquetas distintas. El formato reconoce 17 tags estándar (*ECO*, *Opening*, *TimeControl*, *Termination*, *WhiteTitle*, *BlackTitle*, *WhiteElo*, *BlackElo*, *Event*, *Site*, *White*, *Black*, *Result*, *UTCDate*, *UTCTime*, *WhiteRatingDiff*, *BlackRatingDiff*), un marcador de fin de cabeceras (*SEACABO*, código 00000) y un código de escape para etiquetas no reconocidas (*UNKNOWN.TAG*, código 11111). Cuando el encoder encuentra una etiqueta desconocida, emite *UNKNOWN.TAG* seguido del nombre del tag y su valor como cadenas UTF-8 en crudo, garantizando compatibilidad hacia adelante.

2.1.1. Codificación estructurada por tipo de campo

Campos numéricos Los valores numéricos se representan mediante empaquetado de bits de longitud fija, minimizando la redundancia inherente al formato textual PGN. Entre ellos destacan:

- **Elo de jugadores** (*WhiteElo*, *BlackElo*): codificados en 12 bits.
- **Diferencia de rating** (*WhiteRatingDiff*, *BlackRatingDiff*): representada mediante 1 bit de signo más 9 bits de magnitud.

Cuando un valor no pertenece al rango representable o no es válido, se utiliza un código de escape (*UNKNOWN.TEXT*) seguido de la cadena original en UTF-8, preservando la pérdida cero de información.

Campos categóricos con tabla fija Los campos con un dominio finito y pequeño utilizan tablas de codificación estáticas de longitud fija:

- **Result**: 3 bits (4 valores posibles).
- **Termination**: 3 bits (4 valores posibles).

- `WhiteTitle`, `BlackTitle`: 3 bits cada uno (6 títulos más centinela).

El código ECO se descompone en dos componentes:

- Letra (A--E), codificada en 3 bits mediante tabla fija.
- Número (00--99), almacenado en 7 bits.

Este diseño reduce significativamente el coste respecto a la representación ASCII convencional.

Campos categóricos con codificación Huffman Los campos `Event` y `Opening` presentan un dominio grande pero con distribución de frecuencias muy sesgada: unos pocos valores concentran la mayoría de las apariciones. Por ello se utiliza codificación de Huffman con frecuencias extraídas de las partidas de ejemplo provistas, de modo que los valores más comunes reciben códigos más cortos. El centinela `UNKNOWN_TEXT` se incluye como hoja de baja frecuencia en el árbol, permitiendo escapar a cadena UTF-8 cruda cuando el valor no está en el vocabulario conocido.

El algoritmo de Huffman empleado sigue la construcción estándar de heap de mínimos, pero con dos diferencias respecto a una implementación básica. Los nodos del árbol son estructuras explícitas con referencias a sus hijos y al símbolo asociado, evitando comparaciones implícitas entre objetos heterogéneos. Los códigos se asignan mediante un recorrido en profundidad (DFS) sobre el árbol ya construido, acumulando el prefijo al descender (0 rama izquierda, 1 rama derecha), en lugar de prefijar bits durante la propia fusión. Además, se utiliza un contador de desempate para garantizar que la codificación sea determinista cuando dos nodos tienen la misma frecuencia, asegurando reproducibilidad bit a bit.

Campos temporales (Delta Encoding) Los campos `UTCDate` y `UTCTime` se comprimen mediante codificación diferencial respecto a la partida anterior, explotando la localidad temporal de los datasets de torneos.

`UTCDate` utiliza un esquema de tres niveles:

- **Mismo día:** prefijo 0 (1 bit total).
- **Delta pequeño** $[-63, +63]$: prefijo 10 seguido de 7 bits con offset 63 (9 bits total).
- **Fecha absoluta** (fuera de rango o primera partida): prefijo 11 seguido de la fecha completa en 16 bits (18 bits total).

`UTCTime` utiliza un esquema de dos niveles:

- **Delta pequeño** $[-63, +63]$: prefijo 0 seguido de 7 bits con offset 63 (8 bits total).
- **Tiempo absoluto** (fuera de rango o primera partida): prefijo 1 seguido de 17 bits de segundos del día (18 bits total).

Este esquema permite representar secuencias de partidas consecutivas con alta redundancia temporal de forma altamente eficiente.

Campos de control estructurado El campo `TimeControl` se almacena separando explícitamente:

- tiempo base,
- incremento por jugada.

Ambos valores se codifican mediante enteros de longitud fija, eliminando completamente la representación textual original.

Campos de red y contexto El campo `Site` se optimiza eliminando prefijos redundantes (por ejemplo, <https://lichess.org/>). El identificador restante se codifica mediante Base62, asignando un índice de 6 bits por carácter.

2.1.2. Mecanismos de escape y robustez

CPG5 incorpora un sistema de escape completamente reversible para garantizar pérdida cero de información en campos no contemplados por los esquemas de compresión.

- **Valores desconocidos:** se codifican mediante `UNKNOWN_TEXT`, seguidos de la cadena original en UTF-8.
- **Etiquetas no estándar:** se codifican mediante `UNKNOWN_TAG`, incluyendo tanto el nombre del campo como su valor.

Este mecanismo asegura compatibilidad hacia adelante con extensiones arbitrarias del estándar PGN.

2.1.3. Codificación de nombres de jugadores

La codificación de nombres de jugadores utiliza un esquema híbrido con diccionario incremental e inspiración en la familia LZ, pero con diferencias importantes respecto al LZ78 clásico.

1. **Primera aparición (flag 0):** El nombre se codifica como:

- longitud en 5 bits,
- secuencia de caracteres codificados mediante Huffman sobre un alfabeto de 64 símbolos.

Si el nombre contiene caracteres fuera del alfabeto permitido, se utiliza un mecanismo de escape basado en UTF-8.

2. **Apariciones posteriores (flag 1):** Se sustituye el nombre por su índice en el diccionario global. El número de bits necesarios se ajusta dinámicamente según:

$$\text{máx}(1, \lceil \log_2(N) \rceil)$$

donde N es el tamaño actual del diccionario.

Diferencias con LZ78 En el LZ78 clásico, la unidad de diccionario es una *subcadena*: cada nueva entrada es la concatenación del prefijo más largo ya conocido con el siguiente carácter, y el token emitido es el par (índice de prefijo, carácter nuevo). El diccionario crece con fragmentos de la propia secuencia de entrada.

En el esquema de CPG5, la unidad de diccionario es el *nombre completo del jugador*: la primera vez que aparece se codifica íntegramente con Huffman, y se le asigna un índice en el diccionario; las siguientes apariciones simplemente emiten ese índice. No existe la noción de prefijo parcial ni de extensión carácter a carácter. La similitud con LZ78 es conceptual: ambos mantienen un diccionario incremental que crece con la secuencia de entrada y permiten referenciar entradas anteriores con un índice de ancho creciente. La diferencia clave es la granularidad, que en CPG5 es la palabra completa en lugar del símbolo individual.

2.2. Codificación de Movimientos (Chess Logic)

El núcleo de CPG5 (`encode_game_chess.py`) explota una propiedad fundamental del ajedrez: en cualquier posición dada, únicamente existe un conjunto finito y relativamente pequeño de movimientos legales. En lugar de almacenar la notación algebraica completa de cada jugada, el compresor reconstruye internamente el estado exacto del tablero mediante la biblioteca `python-chess` y almacena únicamente cuál de los movimientos legales disponibles fue seleccionado por el jugador.

Sea N el número de movimientos legales en una posición determinada. Tras generar la lista completa de movimientos legales en un orden determinista, el movimiento ejecutado queda identificado unívocamente por su posición dentro de dicha lista. Por tanto, el coste teórico mínimo para almacenar una jugada es:

$$\lceil \log_2(N) \rceil$$

bits.

Durante la descompresión, el decodificador reconstruye exactamente el mismo tablero, genera nuevamente la lista de movimientos legales y recupera la jugada leyendo únicamente dicho índice. Este mecanismo elimina completamente la necesidad de almacenar coordenadas, piezas, capturas, promociones, jaques o cualquier otro elemento de la notación SAN.

2.2.1. Empaquetado de Plies

Cada medio movimiento (*ply*) se representa mediante dos componentes:

1. El índice del movimiento dentro de la lista de movimientos legales.
2. Un bit adicional (`has_annot`) que indica la presencia o ausencia de anotaciones asociadas.

Para una posición con N movimientos legales, el número de bits necesarios para representar el índice es:

$$w = \begin{cases} 0 & \text{si } N = 1 \\ \lceil \log_2(N) \rceil & \text{si } N > 1 \end{cases}$$

Por consiguiente, el coste total por ply es:

$$\lceil w + 1 \rceil$$

bits.

El flujo de bits resultante se concatena de forma continua para toda la partida y se rellena con ceros únicamente al final del bloque para garantizar alineamiento a byte.

2.2.2. Codificación del Resultado

El resultado final de la partida se almacena separadamente mediante un código fijo de un byte:

Resultado PGN	Código
1-0	0
0-1	1
1/2-1/2	2
*	3

De esta manera se evita almacenar repetidamente la cadena textual completa al final de cada PGN.

2.2.3. Tratamiento de Anotaciones

Las anotaciones PGN encerradas entre llaves (...) se procesan de forma independiente al flujo principal de movimientos.

Durante la compresión, cada bloque de anotación se extrae temporalmente del texto PGN y se sustituye por un marcador interno. Posteriormente, cuando el compresor identifica a qué movimiento pertenece la anotación, activa el bit `has_annot` correspondiente y almacena la información en una sección auxiliar separada.

CPG5 distingue varios tipos de anotaciones frecuentes:

- Evaluaciones de motor (`[%eval ...]`).
- Anuncios de mate (`[%eval #n]`).
- Tiempos de reloj (`[%clk h:mm:ss]`).
- Texto arbitrario no estructurado.

Evaluaciones centipawn Las evaluaciones numéricas producidas por motores de análisis se convierten a centipeones y se almacenan como enteros con signo de 16 bits:

$$\text{eval} = \text{round}(100 \cdot \text{valor})$$

Por ejemplo:

$$+0,35 \mapsto 35$$

$$-1,27 \mapsto -127$$

Esto reduce considerablemente el espacio necesario respecto a la representación textual original.

Anuncios de mate Las evaluaciones de tipo mate:

$$[\%eval\#5]$$

se almacenan mediante un entero con signo de 8 bits que representa directamente la distancia al mate.

Tiempos de reloj Las marcas temporales:

$$[\%clk; h : mm : ss]$$

se empaquetan en un entero de 16 bits utilizando la distribución:

$$4 \text{ bits} + 6 \text{ bits} + 6 \text{ bits}$$

correspondiente a:

- horas (0–15),
- minutos (0–63),
- segundos (0–63).

La codificación ocupa únicamente dos bytes frente a las ocho o más posiciones de la representación textual original.

Texto libre Cuando una anotación contiene información no reconocida por el compresor, se emplea un mecanismo de escape (`TOK_ANNOT_RAW`) que almacena literalmente el contenido original, garantizando una reconstrucción sin pérdida.

Cada secuencia de anotación termina con un byte nulo (`0x00`), permitiendo al decodificador delimitar inequívocamente el final de cada bloque.

2.2.4. Elementos Ignorados

Diversos elementos presentes en el PGN no afectan al estado real del tablero y, por tanto, no necesitan almacenarse explícitamente.

Entre ellos se encuentran:

- números de jugada (1., 2., 15...),
- símbolos NAG (\$1, \$2, etc.),
- variantes entre paréntesis,
- sufijos SAN de valoración (!, ?, !!, ?!, etc.).

Toda esta información puede descartarse sin afectar a la reconstrucción exacta de la secuencia principal de movimientos.

2.2.5. Estructura Binaria de una Partida

Cada partida se almacena como un bloque independiente precedido por su longitud en bytes. Internamente, dicho bloque posee la siguiente organización:

Campo	Tamaño
Número de plies	2 bytes
Resultado	1 byte
Movimientos empaquetados	variable
Anotaciones	variable

El número de plies se almacena mediante un entero sin signo de 16 bits, permitiendo representar hasta 65,535 medios movimientos por partida, muy por encima de cualquier partida práctica.

2.2.6. Formato Global CPG5

Finalmente, el archivo completo utiliza la siguiente estructura:

Campo	Tamaño
Magic Number (CPG5)	4 bytes
Longitud de cabeceras	4 bytes
Cabeceras comprimidas	variable
Número de partidas	4 bytes
Bloques de partidas	variable

La separación explícita entre cabeceras y movimientos permite explotar mecanismos de compresión completamente distintos en ambos dominios. Mientras las cabeceras se benefician de diccionarios, Huffman y codificación diferencial, los movimientos se aproximan al límite de entropía impuesto por las propias reglas del ajedrez, almacenando únicamente la elección realizada entre los movimientos legales disponibles en cada posición.

2.3. Paralelización

Debido a que la compresión inteligente de movimientos requiere la simulación explícita del estado del tablero y la generación de movimientos legales en cada posición, el algoritmo presenta un coste computacional significativamente mayor que el de la compresión de cabeceras. Este proceso depende de forma intensiva de CPU, ya que cada *ply* implica la reconstrucción del conjunto de movimientos legales mediante `python-chess`.

Para mitigar este cuello de botella, CPG5 implementa una estrategia de paralelización multiproceso basada en `concurrent.futures.ProcessPoolExecutor`. Esta elección se debe a que el intérprete de Python limita el paralelismo efectivo mediante el GIL (*Global Interpreter Lock*) en escenarios de cómputo intensivo, haciendo necesario el uso de procesos independientes.

2.3.1. Separación estricta de fases

El pipeline de compresión se divide en dos fases claramente diferenciadas:

- **Compresión de cabeceras:** ejecutada de forma estrictamente secuencial debido a dependencias de estado entre partidas consecutivas, tales como:
 - codificación delta de fechas (`UTCDate`, `UTCTime`),
 - diccionario global de nombres de jugadores (LZ78 incremental).
- **Compresión de movimientos:** completamente independiente entre partidas, lo que permite su paralelización sin riesgos de inconsistencia.

Esta separación es clave: mientras las cabeceras presentan dependencia inter-partida, el bloque de movimientos es funcionalmente puro a nivel de partida individual.

2.3.2. Modelo de paralelización

Las partidas se distribuyen en forma de pares:

[(*i*, movimientos)]

donde *i* representa el índice original de la partida en el archivo.

Cada worker procesa una partida completa mediante la función `_encode_game`, que reconstruye internamente el tablero desde el estado inicial estándar y calcula la secuencia de índices de movimientos legales.

Para optimizar la utilización de CPU y minimizar sobrecarga de IPC, el sistema emplea *chunking* de trabajos (`chunksize=100`), reduciendo el número de mensajes entre procesos.

2.3.3. Ejecución segura y tolerancia a fallos

Cada proceso worker está encapsulado en una función de seguridad:

- Si la compresión de una partida falla, el sistema no aborta el pipeline completo.
- En su lugar, se devuelve un bloque mínimo válido con código de resultado por defecto.
- El error se registra y la partida se marca como degradada.

Este diseño garantiza robustez en datasets de gran tamaño, donde la corrupción parcial de PGN es frecuente.

2.3.4. Reensamblaje determinista

Una vez finalizada la ejecución paralela, los resultados son reordenados utilizando el índice original de cada partida. Este paso es crítico, ya que la ejecución concurrente no garantiza orden de finalización.

El sistema realiza un reensamblaje secuencial en un buffer preasignado:

- Se crea una estructura `move.blocks` indexada por posición.
- Cada resultado del pool de procesos se inserta en su índice correspondiente.
- Posteriormente, se serializa en orden estricto para garantizar reproducibilidad bit-a-bit.

2.3.5. Optimización de escritura

Antes de escribir el archivo final, todos los bloques se acumulan en un `bytearray` contiguo, minimizando operaciones de I/O. Esta estrategia reduce significativamente la fragmentación de escritura y mejora el rendimiento en discos de alta latencia.

2.3.6. Justificación del diseño híbrido

La arquitectura de CPG5 no busca paralelizar todo el pipeline, sino únicamente aquellas secciones donde existe independencia funcional real. En particular:

- Las cabeceras están sujetas a dependencias globales (estado del diccionario y del tiempo).
- Las partidas de ajedrez son independientes entre sí.
- Los movimientos dentro de una partida son secuenciales por naturaleza y no paralelizables sin romper la semántica del juego.

Este enfoque híbrido permite maximizar el rendimiento sin comprometer la exactitud del proceso de compresión ni la reproducibilidad del formato CPG5.

2.4. Estructura del Binario CPG5

El archivo resultante tiene un diseño binario estricto basado en codificación Big-Endian, con separación explícita entre metadatos globales, cabeceras comprimidas y bloques independientes de partidas.

A nivel conceptual, CPG5 no se basa en un flujo continuo de texto comprimido, sino en una estructura híbrida de *bitstream + byte-aligned blocks*, donde cada componente del archivo cumple una función específica dentro del pipeline de decodificación determinista.

- **[4 bytes]** Número mágico identificativo: `b'CPG5'`. Este identificador permite validar la integridad del archivo y evitar interpretaciones erróneas de datos no compatibles.
- **[4 bytes] uint32 (Big-Endian)**: Número total de bits utilizados en el bloque de cabeceras comprimidas. Este valor no representa bytes, sino la longitud exacta del bitstream lógico, lo que permite al decodificador truncar correctamente el padding final.
- **[N bytes]** Bloque de cabeceras comprimidas: Secuencia de bits generada por el módulo de codificación de metadata. Este bloque:

- se empaqueta en bytes mediante alineación a 8 bits,
 - puede contener padding de ceros al final,
 - debe truncarse exactamente a la longitud especificada en el campo anterior.
- [4 bytes] uint32: Número total de partidas M contenidas en el archivo.
 - **Por cada una de las M partidas:**
 - [4 bytes] uint32: Longitud en bytes del bloque completo de la partida (L). Este valor permite al decodificador saltar eficientemente entre partidas sin necesidad de parsing incremental.
 - [L bytes] Bloque de movimientos:
 - [2 bytes] uint16: Número total de *plies* en la partida. Este valor incluye movimientos de ambos jugadores y se utiliza para dimensionar la reconstrucción del tablero.
 - [1 byte] uint8: Resultado de la partida codificado como:
 - ◊ 0 = 1-0
 - ◊ 1 = 0-1
 - ◊ 2 = 1/2-1/2
 - ◊ 3 = *
 - [B bytes] Bitstream de jugadas empaquetadas: Cada *ply* se codifica como:

$$\lceil \log_2(N_{legal}) \rceil + 1$$

bits, donde:

- ◊ los primeros bits representan el índice del movimiento dentro de la lista de movimientos legales ordenada determinísticamente,
- ◊ el último bit indica la presencia de anotación asociada (`has_annot`).

Este bloque se almacena en formato bit-packing y se rellena con ceros únicamente al final del bloque para completar el último byte.

- [A bytes] Bloque de anotaciones:
Secuencia opcional de anotaciones asociadas a aquellos *plies* cuyo bit `has_annot` está activado.

Las anotaciones se almacenan en orden estricto de aparición y se decodifican de forma secuencial mediante tokens tipados:

- ◊ evaluaciones de motor (TOK_ANNOT_EVAL),
- ◊ mates forzados (TOK_ANNOT_MATE),
- ◊ tiempos de reloj (TOK_ANNOT_CLK),
- ◊ anotaciones crudas (TOK_ANNOT_RAW).

Cada anotación finaliza con un byte nulo (0x00), permitiendo delimitación inequívoca durante la decodificación.

2.4.1. Propiedades estructurales del formato

CPG5 adopta un diseño híbrido entre estructura secuencial y acceso indexado:

- El encabezado global es un bitstream comprimido de longitud explícita.
- Cada partida es independiente y autocontenida mediante su longitud en bytes.
- Los movimientos están optimizados como codificación entropía-adyacente al espacio de estados del ajedrez.

Esta estructura permite:

- decodificación streaming parcial,
- acceso directo a partidas individuales,
- paralelización de procesamiento sin dependencias globales,
- y reconstrucción exacta bit-a-bit del archivo original.

2.4.2. Invarianza del formato

El diseño de CPG5 garantiza que cualquier implementación compatible produce exactamente la misma salida binaria, siempre que:

- el orden de movimientos legales sea determinista,
- la codificación de cabeceras respete el mismo estado incremental,
- y el padding de bits se realice exclusivamente al final de cada bloque.

Esto convierte el formato en un sistema completamente reproducible y apto para compresión sin pérdida de datos estructurales del PGN.

3. Resultados

4. Conclusiones

Referencias

- [1] Código fuente provisto del módulo CPG5: `encode_game_chess.py`, `decode_headers.py`, `player_names.py`, `constant_types.py`, `frequencies.py`.
- [2] David A. Huffman, *A Method for the Construction of Minimum-Redundancy Codes*, Proceedings of the IRE, 1952.
- [3] Biblioteca `python-chess`. Implementación de las reglas y generación de movimientos legales.